

Data Governance Copilot

Interview Question Bank

Topic 06 — RAG & Vector Search

12 questions · 4 sections · pgvector · HNSW · Chunking · Embeddings · Ingestion · NullVectorService

Foundational Core concepts **Intermediate** Design decisions **Advanced** Deep internals **Gotcha** Real bugs / traps

RAG Fundamentals & Architecture

Q01 Foundational

What is RAG and why does your Data Governance Copilot need it? What kinds of questions does it answer that pure LLM knowledge cannot?

Hint: Think about the nature of enterprise governance documents and why they aren't in LLM training data.

Key points to cover:

- RAG (Retrieval-Augmented Generation): retrieve relevant document chunks from a vector store, inject into LLM prompt as context
 - LLMs have a training cutoff and never saw your company's internal governance policies, data dictionaries, or incident post-mortems
 - Example questions RAG answers: 'What is the retention policy for PII data per our internal standard?', 'What did the Q3 incident review say about the bookings table?'
 - Without RAG: LLM hallucinates or says 'I don't know' — neither is acceptable in a governance context
 - RAG enables grounded answers: the LLM cites the actual document chunk, not invented policy text
 - KnowledgeAgent is the RAG agent — it queries PGVectorService (or NullVectorService in dev) and returns relevant chunks + sources
 - Governance use case is ideal for RAG: documents are authoritative, relatively stable, and need exact citation not creative interpretation
-

Q02 Foundational

Explain the role of embeddings in your RAG pipeline. What model do you use and why?

Hint: Think about the relationship between query embedding and document chunk embeddings.

Key points to cover:

- Embeddings: dense vector representations of text — semantically similar text produces geometrically close vectors
 - Model: OpenAI text-embedding-3-small — 1536-dimensional vectors, strong multilingual performance, low cost
 - At ingestion: each document chunk is embedded and stored in pgvector alongside its text content
 - At query time: user query is embedded with the same model — produces a vector in the same 1536-dim space
 - Similarity search: find stored chunk vectors with smallest cosine distance to the query vector — most semantically relevant chunks
 - Why text-embedding-3-small over text-embedding-3-large: 5x cheaper, 99% of the quality for typical governance doc retrieval
 - Critical constraint: query and document embeddings MUST use the same model — mixing models produces garbage similarity scores
 - embedder.py has module-level OpenAIEmbeddings import — required for pytest patch() compatibility (module-level import discipline)
-

Q03**Intermediate**

What is PGVectorService vs NullVectorService and how does the IVectorService protocol enable clean switching?

Hint: This follows the same services layer pattern as IDataService — explain the specific vector operations.

Key points to cover:

- IVectorService protocol defines: `similarity_search(query, k, filters) → List[Document]`, `upsert(chunks) → int`
- PGVectorService (real): wraps langchain-postgres PGVector class — connects to Neon, runs HNSW index searches
- NullVectorService (mock): returns empty list for `similarity_search`, no-ops `upsert` — zero external dependencies
- Factory: `ENABLE MOCK=true → NullVectorService`; `ENABLE MOCK=false → PGVectorService(DATABASE_URL)`
- NullVectorService is named 'Null' not 'Mock' deliberately — it's a Null Object pattern, not a test double with assertions
- KnowledgeAgent receives IVectorService in constructor — never imports PGVectorService directly
- Dev benefit: entire RAG pipeline runs locally without Neon DB or OpenAI embeddings — agents still execute, just return empty results
- `core/vector_store.py` is a backward-compat shim — delegates to `services/pgvector/` — kept so old import paths don't break

HNSW Indexing & Similarity Search

Q04**Intermediate**

What is HNSW and why did you choose it over IVFFlat for your pgvector index?

Hint: Both are approximate nearest neighbour algorithms — explain the key tradeoffs.

Key points to cover:

- HNSW (Hierarchical Navigable Small World): graph-based ANN index — builds a multi-layer proximity graph
- IVFFlat (Inverted File with Flat quantisation): cluster-based — partitions vectors into Voronoi cells, searches nearest clusters
- HNSW advantages: better recall at same speed, no training phase required, handles incremental inserts well
- IVFFlat requires a training step (`CREATE INDEX` takes long for large datasets) and recall degrades with inserts between retrains
- Governance document corpus is relatively small (hundreds to low thousands of chunks) — HNSW build time is negligible
- HNSW query time: $O(\log N)$ — fast even without GPU, suitable for Neon serverless PostgreSQL
- `init_neon.sql` has HNSW index creation commented out — to be run after first ingestion (no vectors = index build fails)
- pgvector HNSW params: `m=16` (connections per layer), `ef_construction=64` (build quality) — defaults are fine for this corpus size

```
-- init_neon.sql HNSW index (run after first ingestion) CREATE INDEX ON document_embeddings USING
hsw (embedding vector_cosine_ops) WITH (m = 16, ef_construction = 64); -- For query-time recall
tuning: SET hsw.ef_search = 40; -- higher = better recall, slower
```

Q05**Advanced**

Your similarity search uses cosine distance. When would you switch to L2 (Euclidean) or inner product, and what does that mean for how embeddings are stored?

Hint: This tests depth of understanding of vector similarity metrics.

Key points to cover:

- Cosine distance: measures angle between vectors — ignores magnitude, only direction matters
- Best for: text embeddings where magnitude varies by document length — cosine normalises this away
- L2 (Euclidean): measures straight-line distance — magnitude matters; two identical texts with different lengths would have different L2 distances
- Inner product: cosine × magnitude — best when embeddings are explicitly trained with inner product objective (e.g. some bi-encoders)
- OpenAI text-embedding-3-small is trained for cosine similarity — using L2 would give subtly wrong rankings
- pgvector operator: `vector_cosine_ops` for cosine, `vector_l2_ops` for L2, `vector_ip_ops` for inner product
- Storage impact: HNSW index is built for ONE operator — changing metric requires dropping and rebuilding the index
- Normalised embeddings (unit vectors): cosine and inner product become equivalent — OpenAI embeddings are L2-normalised by default

Q06**Advanced**

How does your similarity search handle metadata filtering? For example, only search chunks from documents tagged with 'retention' data product.

Hint: pgvector supports WHERE clause filtering alongside vector search.

Key points to cover:

- `PGVectorService.similarity_search(query, k, filters={'data_product': 'retention'})` passes filters as SQL WHERE conditions
- langchain-postgres PGVector supports pre-filtering: vector search + metadata filter in one query
- Document chunks store metadata JSON column: `{'source': 'policy_v3.pdf', 'data_product': 'retention', 'page': 4}`
- Pre-filter vs post-filter: pre-filter (WHERE first, then HNSW search) can miss results if filter is very selective
- Post-filter (HNSW search on all vectors, then filter results): better recall but returns fewer than k results after filtering
- For governance use case: pre-filter is correct — user asking about retention policy should only see retention-tagged chunks
- `chunker.py` infers `data_product` from filename during ingestion — `'retention_policy_2024.pdf' → data_product='retention'`
- Filter is passed from `AgentState.data_products` — supervisor's intent classification extracts which products are relevant

Chunking Strategy & Ingestion Pipeline

Q07**Intermediate**

Explain your chunking strategy: RecursiveCharacterTextSplitter with chunk_size=512 and chunk_overlap=64. Why these specific values?

Hint: Chunk size is one of the most impactful RAG hyperparameters — justify your choices.

Key points to cover:

- RecursiveCharacterTextSplitter: splits on paragraph → sentence → word boundaries in order — preserves semantic units
- chunk_size=512 characters: roughly 100-150 tokens — fits well within embedding model context and retrieval precision sweet spot
- Too large (e.g. 2048 chars): chunk contains multiple topics — embedding averages over them, retrieval becomes imprecise
- Too small (e.g. 128 chars): chunk lacks context — retrieved snippet is meaningless without surrounding text
- chunk_overlap=64: 12% overlap — ensures sentences spanning chunk boundaries are captured in at least one chunk
- Without overlap: a key sentence at the end of chunk N and start of chunk N+1 might not be retrieved as a unit
- 512/64 is a widely validated default for enterprise document RAG — production-safe starting point
- Improvement: semantic chunking (split at topic boundaries, not character count) — better for long policy documents with clear sections

```
# chunker.py from langchain.text_splitter import RecursiveCharacterTextSplitter splitter =
RecursiveCharacterTextSplitter( chunk_size=512, chunk_overlap=64, separators=['\n\n', '\n', '. ', '
', ''], ) def chunk_document(text, metadata): chunks = splitter.create_documents([text],
metadatas=[metadata]) for chunk in chunks: chunk.metadata['content_hash'] =
sha256(chunk.page_content) return chunks
```

Q08**Intermediate**

Walk through the full ingestion pipeline from a PDF file to a searchable vector in pgvector.

Hint: There are four stages — loaders, chunker, embedder, store — explain each.

Key points to cover:

- Stage 1 — loaders.py: LangChain document loaders at module level; load_document(path) detects file type (PDF/DOCX/PPTX/Excel/text) and returns List[Document]
- PDFs: PyPDFLoader extracts text page by page; DOCX: Docx2txtLoader; PPTX: UnstructuredPowerPointLoader; Excel: UnstructuredExcelLoader
- Stage 2 — chunker.py: RecursiveCharacterTextSplitter splits each Document into 512-char chunks with 64-char overlap; adds content_hash + inferred data_product to metadata
- Stage 3 — embedder.py: OpenAIEmbeddings(text-embedding-3-small) embeds chunks in batches of 100 — batch=100 reduces API round trips
- Stage 4 — store.py: psycopg2 connects to Neon (sslmode=require); SHA-256 content_hash deduplication check before upsert; PGVector.add_documents() inserts chunks + embeddings
- Deduplication: if SHA-256 of chunk text already exists in DB, skip upsert — prevents duplicate embeddings from re-ingesting the same doc
- Trigger paths: POST /ingest (FastAPI multipart) → Airflow on_demand DAG; FileSensor DAG watches docs/ folder; MCP tool ingest_document_url downloads and ingests
- All imports at module level — critical for pytest patch.object() to intercept psycopg2.connect and OpenAIEmbeddings

Q09**Gotcha**

In `test_day19.py` the ingestion tests use `patch.object(store_module.psycpg2, 'connect', ...)` instead of `patch('psycpg2.connect')`. Why does this matter?

Hint: This is the module-level import discipline gotcha — a recurring bug theme in the project.

Key points to cover:

- `store.py` imports `psycpg2` at module level: `import psycpg2` (not inside a function)
- `patch('psycpg2.connect')` patches the `psycpg2` module in the `psycpg2` package namespace
- But `store.py` has already bound its own reference to `psycpg2` at import time — patching the original doesn't affect the stored reference
- `patch.object(store_module.psycpg2, 'connect')` patches the `psycpg2` object AS SEEN BY `store_module` — the correct reference
- This is the standard Python mock gotcha: patch where it's used, not where it's defined
- Same principle applies to `embedder.py` (`OpenAIEmbeddings`) and `loaders.py` (`PyPDFLoader` etc.)
- Bug #39-41 in the project: all three ingestion modules had imports inside functions — moved to module level precisely to enable this `patch.object` pattern
- Rule: if you can't `patch.object(module.SomeClass)`, the import is probably inside a function — move it to module level

Production Considerations & Design Depth

Q10**Advanced**

Your project uses a single pgvector table for all document embeddings. What are the scaling and isolation problems with this and how would you fix them?

Hint: Think about multi-tenancy, corpus separation, and index performance.

Key points to cover:

- Single table problem 1: all data products (retention, bookings, CAC, LTV) share one vector namespace — similarity search returns chunks from unrelated products unless filtered
- Single table problem 2: index size grows with all documents — HNSW build time and memory increase even for rarely-queried products
- Single table problem 3: no tenant isolation — in a multi-org deployment, org A could theoretically retrieve org B's documents
- Fix 1: one table per data product — `'embeddings_retention'`, `'embeddings_bookings'` — each with its own HNSW index
- Fix 2: collection-based partitioning — langchain-postgres PGVector supports named collections; each collection is a logical namespace
- Fix 3: row-level security (RLS) — PostgreSQL RLS policies on the embeddings table based on `tenant_id` in metadata
- Fix 4: separate pgvector instance from transactional DB — your P0 item mentions this; OLTP workload and vector HNSW scans compete for I/O
- Current design is fine for single-org, small corpus — acknowledge the limitation and describe the evolution path

Q11**Advanced**

A user asks a question that is semantically ambiguous — the top 3 retrieved chunks are from different documents with contradictory governance policies. How should your system handle this?

Hint: This is about RAG answer quality, not just retrieval mechanics.

Key points to cover:

- Current behaviour: KnowledgeAgent returns top-k chunks; synthesizer_node includes them all in context — LLM may pick one arbitrarily
 - Problem: LLM might silently favour one policy over another without flagging the contradiction to the user
 - Fix 1: confidence scoring — if top-k chunks have similar cosine distances but contradictory metadata (different doc dates), flag for review
 - Fix 2: source metadata in response — always surface document name, version, and date in sources list so user can assess recency
 - Fix 3: explicit conflict detection in synthesizer prompt — instruct LLM: 'If retrieved chunks contradict each other, state the contradiction explicitly'
 - Fix 4: document versioning — mark superseded policies as inactive in metadata; filter out inactive documents before similarity search
 - Fix 5: HITL trigger — if KnowledgeAgent detects conflicting sources, set pending_action requiring human review before returning answer
 - This is a governance-critical scenario — an incorrect policy answer has real compliance consequences
-

Q12**Design**

Your embedding model is OpenAI text-embedding-3-small. The business decides to switch to a self-hosted model for data privacy. What breaks and what do you need to rebuild?

Hint: This is an embedding model migration — understand the full blast radius.

Key points to cover:

- Critical constraint: ALL existing embeddings in pgvector were created with text-embedding-3-small (1536 dims) — they are incompatible with a new model
 - New model may have different dimensionality (e.g. 768 dims for a BERT-based model) — the pgvector column type and HNSW index would need to change
 - Even same-dimension models: cosine distance between old and new embeddings is meaningless — mixed-model search returns garbage
 - Required steps: 1) deploy new embedding model (HuggingFace on SageMaker or local Ollama); 2) re-embed ALL documents; 3) drop and recreate pgvector table + HNSW index; 4) update embedder.py to use new model class
 - Zero-downtime migration: dual-write period — new documents embedded with new model; old documents served from old table; cutover once re-embedding complete
 - embedder.py abstraction: replace OpenAIEmbeddings with HuggingFaceEmbeddings or a custom class — same interface, different backend
 - Privacy benefit: embeddings never leave your VPC — no document text sent to OpenAI API
 - Cost benefit: self-hosted model has zero per-token cost — relevant for large corpus re-ingestion
-